

Comparing Encoding Architectures for Supervised Contrastive Learning in Tabular Classification

1st Preston Billion-Polak
Florida Atlantic University
Boca Raton, FL, United States
pbillionpola2019@fau.edu

2nd Taghi M. Khoshgoftaar
Florida Atlantic University
Boca Raton, FL, United States
khoshgof@fau.edu

Abstract—Contrastive learning has proven to be an effective classification approach in complex and challenging machine learning tasks, especially those with limited class labels such as self-supervised and low-shot learning, and those with high-signal data modalities such as text and images. However, it is less examined in more traditional classification tasks which are fully supervised, have only two classes, and often feature tabular and mostly numeric data. Despite their simplicity, these tasks are both challenging—especially when class imbalance is present—and highly prevalent in the real-world, such as in fraud detection for financial transactions, network activity, or insurance claims. While previous work has established a baseline performance for contrastive learning methods in these tasks, these works are relatively scarce, and exploration and optimization remain limited overall. In this work, we evaluate a Siamese network on a fully-supervised big-data binary classification task—a Medicare Part D claims dataset—using three different encoding architectures: a Recurrent Neural Network (RNN), a 1D Convolutional Neural Network (CNN), and a Multi-Layer Perceptron (MLP). We conduct manual hyperparameter tuning, ten runs of five-fold cross-validation, and statistical tests to reinforce the validity of our results. We find that the MLP and CNN encoders significantly outperform the RNN encoder across all configurations, but that performance variation from hyperparameter tuning is comparatively small. In addition, we find that both Siamese CNNs and MLPs highly outperform GBDT methods commonly used on this dataset, suggesting a strong alternative for traditional classification tasks.

Index Terms—Contrastive Learning, Supervised Contrastive Learning, Siamese networks, Data Representations, Metric-Based Learning, Fraud Detection, Recurrent Networks

I. INTRODUCTION

Contrastive learning [1] is an increasingly popular framework for challenging classification tasks such as self-supervised, low-shot, or multimodal learning. Fundamentally, these frameworks operate by reframing a classification task from a decision on a single instance to a decision on multiple instances. For example, in the case of image classification, a traditional classifier may be trained to predict whether a given image shows a dog, cat, or rabbit. In contrast, a contrastive-learning-based model would instead take as input a pair of images, and predict whether they show the same type of animal (i.e., they belong to the same class). Despite the apparent simplification in reducing a multi-class problem to a two-class problem (similar/dissimilar), contrastive learning approaches have seen great success in challenging machine learning tasks, especially those with limited data conditions [2]

Contrastive learning methods generally learn a mapping function which transforms an input instance into a different feature space, optimized such that similar instances are clustered together, while dissimilar instances have high distance. In particular, Siamese networks [3] (the typical implementation of fully-supervised contrastive learning) generally use a single encoding model, often a neural network, to uniformly transform each input instance, and optimize this model through standard backpropagation. However, despite the increasingly large literature using contrastive learning methods, there is little work investigating the encoder model as an experimental factor in Siamese frameworks. Instead, we find that most studies instead use only a single encoder model, with little thought put into optimizing its architecture or parameters to improve overall discrimination ability.

In this work, we aim to conduct an exploratory comparison of three different encoding network architectures—a Convolutional Neural Network (CNN), an Recurrent Neural Network (RNN), and a Multi-Layer Perceptron (MLP)—in a Siamese network, to determine their impact on model performance. We aim to evaluate models on a classification task which is relatively simple, so as not to introduce any confounding factors, but still challenging, so as to properly showcase any differences in the capabilities of each Siamese network. To this end, we use a publicly available Medicare Part D fraud classification dataset, which is binary-class, fully-labeled, and tabular, but poses challenges in the form of data size (>5,300,000 instances) and especially class imbalance (<4,000 positive instances). To ensure the robustness of our results, we systematically vary model hyperparameters for each encoder architecture, compare results to those of strong, non-contrastive Gradient-Boosted Decision Tree (GBDT) methods, and use statistical tests such as Analysis of Variance (ANOVA) to analyze our results.

The remainder of this paper is structured as follows. Section II describes works adjacent to our own, including works which give more attention to encoder choice, or which use the encoding architectures which we test in our work. Section III explains Siamese networks and our implementation of the framework, as well as our implementations of the three encoder architectures used (including the experimental parameters varied for each). Section IV goes into detail on our experimental setup, including our dataset and specific model

evaluation process. Section V shows and discusses our results, including performance metrics and analysis outcomes. Finally, Section VI concludes the paper.

II. RELATED WORK

Siamese networks [3] are a specific type of contrastive learning model which use a single “encoder” model to process inputs into an alternative feature space; a simple prediction head is then used to determine the similarity of two encoded instances. The encoder is optimized so that this similarity is high for instances of the same class, and low for instances of differing classes. Some works frame this procedure as using two “synced” encoder models with tied weights, though in practice this is equivalent to a single encoder model.

Siamese networks can be used either for supervised contrastive learning, where similar and dissimilar pairs are constructed from the input data using class labels, or for self-supervised contrastive learning, where similar and dissimilar pairs are generated from unlabeled inputs using data augmentation techniques. In this work, we examine three alternative architectures for Siamese network encoder models: a CNN, an RNN, and an MLP, each of which has seen considerable individual use in prior contrastive learning frameworks; we provide select examples of each below. While other approaches to contrastive learning exist, we restrict our comparison to Siamese networks for their prevalence in fully-supervised settings, and their reliance on a single unified encoder model.

Contrastive learning is commonly used for image- or video-based tasks such as object detection [2]. Fully-connected CNNs are particularly well-equipped to extract information from these data types, and have accordingly been used as encoders in Siamese networks for change detection in aerial images [4], diagnosis from medical scans [5], and various other vision tasks [6]. 1-dimensional CNNs, such as the one used in our study, are less common, but have been used in Siamese networks before, such as by Chatterjee et al. [7], who use this approach for anomaly detection in physical activity time-series data.

Similarly, for text-based or time-series data, RNNs are well-suited architectures, and have accordingly been used as encoders for various Siamese models. Fernández-Llaneza et al. [8] apply a Siamese network with Long Short-Term Memory (LSTM) encoders to classify compounds as bioactive or inactive; Mueller and Thyagarajan [9] use a Siamese LSTM model to predict natural-language sentence similarity; and Neculoiu et al. [10] use a similar model to predict occupational title groupings.

Finally, MLPs, also known as fully-connected neural networks, are a relatively simple but ubiquitous machine learning approach. However, they are most commonly applied to tabular data, whereas contrastive learning has been increasingly popular in tasks with more complex modalities, and thus MLPs as Siamese encoders appear to be less common than CNNs or RNNs. Zhao et al. [11] apply a Siamese framework which encodes instances using a combination of two different MLPs (one shallow, one deep), and apply this model for software

defect prediction; and Zheng et al. [12] use a Siamese MLP for face identification, and are additionally able to use the learned feature space as a data visualization tool.

Naturally, each of these works chooses a single encoder model suited for the task at hand, but does not investigate or compare multiple encoder architectures or model parameters. As an exception, Chen et al. [13] propose a contrastive learning framework named MoCo v3, evaluating its performance, size, and robustness in various object detection and image recognition tasks. In one of these experiments, the encoder for MoCo v3 and three other contrastive frameworks is varied between a ResNet-50 (a popular CNN model) and Vision Transformer (ViT) [14] models (a much more recent architecture for vision tasks) of different sizes. They find that the encoder architecture makes a significant impact on performance; using a ViT with a comparable parameter count to the ResNet underperformed somewhat, but a ViT with $\sim 3\times$ the parameters performed better or on par across frameworks. Khosla et al. [15] similarly compare their proposed SupCon loss in a supervised contrastive learning framework with three ResNet size variants as encoders; however, differences in performance from this encoder variation were not discussed, with the primary analytical focus again being put on the proposed model. We also note that only convolutional ResNet encoders were tested, with no examination of other architecture styles.

After conducting a thorough literature search, to the best of our knowledge, Chen et al. [13] is the only work in the machine learning literature to make a structured comparison of distinct encoding architecture types within the same contrastive frameworks. However, Chen et al.’s work aims to establish the proposed framework’s superiority to other contrastive learning methods, and benchmarking different encoder architectures is not its primary motivation. This motivates our work to conduct an explicit, expanded comparison of a wider variety of architectures within a single framework. We are also the first work to conduct a comparison such as this for fully-supervised, rather than self-supervised, contrastive learning; this represents a comparatively simpler but overall more common learning environment, and removes the need for data augmentation or pair generation techniques which may confound results.

III. METHODOLOGY

A. Siamese Architecture

To optimally compare the performance of different model architectures for Siamese encoders, we aim to use a model framework with minimal additions to the basic Siamese network template. To this end, we base our model framework off of Fernández-Llaneza’s SiameseCHEM [8] model, the source code of which was published on GitHub¹. SiameseCHEM was originally designed to classify molecules, represented textually as “SMILES” strings, as bioactive or inactive. Relevant pre-processing steps and other additional functionalities were relatively simple to disable, leaving only a relatively basic

¹<https://github.com/MolecularAI/Siamese-RNN-Self-Attention>

Siamese network template. This framework takes a labeled input dataset, constructs a balanced set of pairs (50% similar, 50% dissimilar), passes each instance in a pair through the encoder model, computes the cosine similarity of the resulting encodings, and optimizes the encoder based on this similarity.

B. Encoders

For the MLP encoder, we experimentally vary the number of hidden layers n , the number of nodes in each hidden layer h , and the number of nodes in the output layer o . In other terms, o represents the dimensionality of the encodings produced by the MLP which are used for comparison. We vary $n \in \{0, 1, 2, 3\}$, $h \in \{16, 48\}$, and $o \in \{64, 128, 256\}$. A larger range of parameters was explored in initial testing; however, we restrict discussion of our results to this smaller set of well-performing values for space limitations. This is also the case for the CNN and RNN encoders discussed in the following sections. We use a simple Mean Squared Error (MSE) loss function for both this encoder and the CNN.

For the CNN encoder, we stack one-dimensional convolutional layers with a specified number of channels; the last of these layers compresses the input back into a single data channel. This is followed by a flattening operation, then a single fully-connected layer to put the convolved data into a specified output dimensionality. We experimentally vary the number of hidden convolutional layers $n \in \{0, 1, 2\}$, the number of data channels in the hidden representations $c \in \{8, 16, 32\}$ (when applicable), the convolution kernel size $k \in \{3, 5, 9\}$, and finally the number of output dimensions from the final layer in $o \in \{64, 96, 128\}$.

Finally, for the RNN encoder, we implement extra steps to allow for the use of tabular data. More specifically, we follow the steps of [16], which evaluates a similar Siamese recurrent network on a similar tabular dataset, and implement a simple binning process to convert our numeric data into a series of discrete tokens. This process involves normalizing each feature to have a standard deviation of 1.0 and a minimum value of 1.0, to ensure all values have a more consistent range and are all positive. Each feature value is then multiplied by a given “data scaling” constant s , before being truncated to an integer token. Larger values of s result in a larger range of token values, which allows for more “precise” bins and more informative tokens; however, the total range of token values is also increased, requiring a larger dictionary.

After this tokenization process, we choose to follow many of the design decisions chosen by Fernández-Llaneza et al. [8], who use the PyTorch Embedding module [17] to process each token into a random 128-dimensional embedding before passing the sequence into their bidirectional RNN model. Also as per [8], we use a log-cosh loss function and clamp the cosine-distance similarity to a range of $[0, \infty)$, but we do not use their implemented self-attention mechanism. Both the self-attention mechanism and other loss/clamp configurations were tested in preliminary experiments, but these did not yield noteworthy performance compared to the final configurations used. These results are omitted for space limitations.

We vary the cell type used in the RNN $c \in \{\text{GRU}, \text{LSTM}\}$, the data scaling factor $s \in \{1.5, 2.0, 2.5\}$, and the output dimensionality $o \in \{128, 256, 512\}$. Gated Recurrent Units (GRUs) are an alternative cell type for RNNs which are considered simpler than Long Short-Term Memory (LSTM) units; while Fernández-Llaneza present results from a bidirectional LSTM network, we also test a bidirectional GRU network, as per previous evaluations of recurrent Siamese models in similar settings [16], [18].

IV. EXPERIMENTAL DESIGN

A. Dataset

In this work, we aim to compare Siamese encoder architectures in a maximally straightforward classification setting; that is, a fully-supervised task with fully tabular, numeric data. However, we also aim for a task which is challenging enough to effectively highlight large differences in model performance. To this end, we use a highly-imbalanced fraud detection task constructed from publicly available Medicare Part D claims data. This data has been used in prior work to train and evaluate Siamese architectures [18], and in the more general tabular-data classification literature [19], [20].

We follow the data preparation steps of various previous works, including data aggregation by National Provider Identity, labeling using the public List of Excluded Individuals and Entities, encoding of categorical variables using target encoding, and removal of uninformative features. We omit further detail for space considerations; the reader may refer to [18] for more information. This results in a final dataset of ~ 5.3 million claims, of which $\sim 0.07\%$ represent known fraudulent activity; this represents an extreme class imbalance. Imbalance of this severity is known to pose significant challenges to most machine learning models [21], [22]; while mitigation approaches such as data sampling can alleviate the issue, these can result in significant overfitting risks or information loss. Accordingly, most works evaluating machine learning models for classification on this dataset report high but imperfect results (AUPRC scores ≤ 0.80), despite its apparent simplicity.

B. Model Evaluation

Before model training and evaluation, Random Undersampling (RUS) was applied to training data in order to reduce the severe class imbalance present in our dataset. For the results presented in this work, a target ratio of 1:9 is used for all Siamese model configurations. This ratio provided the strongest (or statistically similar to the strongest) classification performance for all three encoders. Due to space limitations, detailed results for each sampling ratio used for each model are not shown.

We evaluate the performance of all models using Area Under the Precision-Recall Curve (AUPRC), representing a model’s discrimination ability between both the positive and negative class (i.e., similar and dissimilar pairs) across all classification thresholds. This threshold-independence allows AUPRC to capture more information than threshold-dependent metrics such as F1 score; additionally, AUPRC has been

shown to be more reliable under imbalanced conditions than similar metrics such as AUC-ROC [23]. AUPRC scores have a minimum value of the prior probability of the positive class (in this case, 0.0007) for a model which classifies instances completely randomly, and a maximum value of 1.0 for a model which perfectly discriminates all instances.

Following the steps of [18], we also evaluate and report the performance of two Gradient-Boosted Decision Tree (GBDT) models (CatBoost [24] and XGBoost [25]) on this dataset. These models are commonly used for imbalanced supervised classification tasks such as fraud detection [26], and GBDTs in general are considered strong performers for tabular classification tasks [27]. Like with the Siamese models, we again conduct hyperparameter tuning on factors such as the number of trees and RUS sampling ratio used; however, for space, we do not present results for all configurations, and instead provide AUPRC scores for only the best-performing configurations (averaged over ten runs of five-fold cross-validation).

We evaluate all models under ten runs of five-fold cross-validation, for 50 total trained models per configuration. All AUPRC scores are averaged over these 50 runs, to mitigate any effects of random chance in data sampling or model training. After reporting aggregated AUPRC results, we perform Analysis of Variance (ANOVA) for each encoder architecture, showing which experimental factors have a statistically significant impact on model performance. We also conduct Tukey’s Honestly Significant Difference (HSD) tests to compare the Siamese models with baseline models; this test produces “groupings” of values which belong to different bands of performance. The top-performing values are put into the group ‘a’, the values significantly worse than these put into group ‘b’, values significantly worse than the group ‘b’ values are placed in group ‘c’, and so on.

V. RESULTS & ANALYSIS

A. Performance Results

We present AUPRC results for various configurations under ten runs of five-fold cross-validation for the MLP, CNN, and RNN encoders in Tables I–III. Values in boldface indicate the maximum for the relevant table areas.

TABLE I
AUPRC SCORES FOR MLP ENCODING ACROSS CONFIGURATIONS.

AUPRC		Output Dimensions			Average
# Layers	Layer Size	64	128	256	
None	—	0.91789	0.92034	0.92061	0.91962
1	16	0.92999	0.92971	0.93002	0.92991
	48	0.92772	0.92869	0.92934	0.92858
2	16	0.92824	0.92729	0.92793	0.92782
	48	0.92366	0.92398	0.92501	0.92422
3	16	0.92549	0.92805	0.92936	0.92763
	48	0.92452	0.92389	0.92185	0.92342
Average		0.92536	0.92600	0.92630	0.92588

The MLP encoder provides an average AUPRC score of 0.92588, with the best single performance of 0.93002 resulting from the configuration with a single hidden layer of 16 nodes,

with a final output encoding size of 256 dimensions. The worst single performance of 0.91789 is produced by the simplest configuration, with no hidden layers and only 64 dimensions.

The CNN encoder provides an average AUPRC score of 0.92467, and a peak score of 0.93434 from the configuration with 2 hidden convolutional layers with a 32-channel representation, a kernel size of 9, and a final encoding size of 96 dimensions. The worst single performance of 0.91152 is yielded when no hidden layers are used, with a kernel size of 9 and a 96-dimensional output.

Finally, the RNN encoder provides a much lower average AUPRC score of 0.75507, roughly consistent with prior evaluations of recurrent Siamese models on this dataset [18]; the best score of 0.78607 is found when the GRU cell type is used, with a data scaling factor of 2.5 and an output dimensionality of 128. The worst performance yielded by this encoder, and the worst performance we found from our Siamese models overall, is an AUPRC score of 0.72730, achieved with the LSTM cell type, a scaling factor of 2.0, and an output dimensionality of 128.

TABLE II
AUPRC SCORES FOR RNN ENCODING ACROSS CONFIGURATIONS.

AUPRC		Output Dimensions			Average
Cell Type	Data Scale	128	256	512	
LSTM	1.5	0.74499	0.76008	0.75027	0.75178
	2.0	0.72730	0.73668	0.73566	0.73322
	2.5	0.74329	0.73144	0.73927	0.73800
GRU	1.5	0.77601	0.77762	0.76639	0.77334
	2.0	0.75354	0.76187	0.75719	0.75351
	2.5	0.78607	0.78540	0.75809	0.77652
Average		0.75520	0.75885	0.75115	0.75507

In total, the MLP encoder has the highest average AUPRC across parameter configurations (by a margin of 0.0012 AUPRC), while the CNN encoder achieves the highest peak performance for a single configuration (by a margin of 0.0043 AUPRC). We note that these margins are quite small in comparison to the range of performance values across parameter configurations (0.01–0.02), indicating that these architectures are nearly identical in performance on this task when optimized. In contrast, the RNN encoder performs significantly worse than the other two, to a degree where parameter choice almost certainly does not play a meaningful role.

B. Statistical Analyses & Discussion

We provide ANOVA results in Tables IV–VI. For the MLP encoder, it can be seen that the number of layers and the size of each layer is highly significant ($p \ll 0.05$), whereas the number of output dimensions does not have a significant impact within the tested range. The optimal values for these significant features are relatively apparent, with a single hidden layer and a smaller layer size being straightforwardly optimal. Thus, we do not conduct Tukey’s HSD test to explicitly determine performance groupings for these parameters. Overall, we find that a relatively simple MLP works best for encoding this data efficiently.

TABLE III
AUPRC SCORES FOR CNN ENCODING ACROSS CONFIGURATIONS.

AUPRC	Output Dims.	64			96			128			
# Layers	Channels\Kernel	3	5	9	3	5	9	3	5	9	Average
None	—	0.91368	0.91517	0.91216	0.91423	0.91507	0.91152	0.91386	0.91670	0.91181	0.91374
1	8	0.92330	0.92067	0.92311	0.92539	0.92229	0.92441	0.92448	0.92199	0.92413	0.92326
	16	0.91865	0.92706	0.92703	0.91693	0.92644	0.92881	0.91774	0.92602	0.92647	0.92401
	32	0.92383	0.92589	0.92749	0.92457	0.92519	0.92942	0.92647	0.92922	0.92918	0.92681
2	8	0.92722	0.92586	0.92905	0.92646	0.92592	0.92828	0.92856	0.92648	0.92841	0.92727
	16	0.92971	0.92858	0.92994	0.92991	0.92784	0.93154	0.93096	0.93110	0.93398	0.93007
	32	0.92894	0.92968	0.93238	0.92679	0.93045	0.93434	0.92978	0.93210	0.93376	0.93091
Average		0.92316	0.92419	0.92520	0.92310	0.92423	0.92607	0.92455	0.92623	0.92682	0.92467

For the CNN encoder, we find that all four factors (number of hidden layers, number of hidden channels, kernel size, and output dimensionality) are highly significant. In comparison to the MLP model, different patterns emerge when examining optimal parameter values. In this case, a higher model complexity appears to be optimal, with more hidden layers, more hidden channels, and a higher output dimensionality yielding generally improved performance. A lower output dimensionality of 64 is optimal when no hidden layers are used, potentially because no intermediate high-dimensional input representation is formed, and thus higher output dimensionalities become overly sparse. The highest kernel size of 9 is also optimal, though interestingly the lowest size of 3 consistently outperforms the median value of 5.

Finally, for the RNN encoder, all experimental factors are again found to be significant within the tested ranges, with cell type being the most influential overall. Interestingly, higher data scaling factors (and therefore more granular token dictionaries) appear to benefit GRU-based models, but harm LSTM-based models. Additionally, in comparison to MLP and CNN encoders, where higher output dimensionality had a small but positive effect, the ideal value for the RNN encoder is the median value tested of 256.

TABLE IV
THREE-FACTOR ANOVA RESULTS FOR MLP ENCODER

MLP	Df	Sum Sq.	Mean Sq.	F-Value	Pr(>F)
# Layers	3	93.2	31.081	21.046	0.0000
Layer Size	1	20.9	20.872	14.133	0.0002
Output Dims.	2	1.6	0.810	0.548	0.5781
Residual	1043	1540.3	1.477		

TABLE V
THREE-FACTOR ANOVA RESULTS FOR RNN ENCODER

RNN	Df	Sum Sq.	Mean Sq.	F-Value	Pr(>F)
Cell Type	1	1781	1780.7	480.82	0.0000
Data Scale	2	89	44.5	12.02	0.0000
Output Dims.	2	465	232.3	62.73	0.0000
Residual	894	3331	3.7		

C. Comparison to GBDT Baselines

For a final comparison, we present a direct comparison of the optimal configurations of each of the three Siamese

TABLE VI
FOUR-FACTOR ANOVA RESULTS FOR CNN ENCODER

CNN	Df	Sum Sq.	Mean Sq.	F-Value	Pr(>F)
# Layers	2	1297	648.3	463.266	0.0000
Output Dims.	2	13	6.5	4.654	0.0095
# Channels	2	69	34.5	24.659	0.0000
Kernel Size	2	47	23.4	16.707	0.0000
Residual	4641	6495	1.4		

models, plus CatBoost and XGBoost models, in Table VII. We use Tukey’s HSD test to determine which models are statistically distinguishable from each other. We find that Siamese models using CNN or MLP encoders outperform both GBDT models by a wide margin, while the model using an RNN encoder underperforms CatBoost but outperforms XGBoost. We also note a significant difference in size: while the MLP and CNN encoders have on the order of 10^4 trainable parameters, the RNN encoder has on the order of 10^6 ; thus, we strongly recommend against the RNN encoder in this task.

TABLE VII
TUKEY’S HSD GROUPINGS FOR EACH MODEL WITH OPTIMAL r

Model	Top AUPRC	HSD Group
Siamese – CNN	0.93434	‘a’
Siamese – MLP	0.93002	‘a’
CatBoost – 1:81	0.80271	‘b’
Siamese – RNN	0.78607	‘c’
XGBoost – No RUS	0.74923	‘d’

VI. CONCLUSIONS

In this work, we compare the performance of Siamese networks using MLP, CNN, and RNN encoders on a fully-supervised, highly-imbalanced fraud detection task. To the best of our knowledge, we are one of the first works to conduct an experimental performance comparison of encoder architectures in a Siamese framework, and the first work to do so in a fully-supervised environment. We find that the MLP and CNN encoders significantly outperform two GBDT baselines considered to be state-of-the-art in this type of task [27], while the RNN encoder only performs on par with them, despite using many more parameters than the other encoders. While encoder architecture had a significant effect on performance, as confirmed by statistical tests under multiple

runs of cross-validation, other varied model parameters such as encoding dimensionality had comparatively little impact, suggesting that manual hyperparameter optimization may not be crucial for Siamese networks. We strongly encourage future work to perform similar encoder comparisons in different data domains and modalities, under more contemporary contrastive frameworks, and using additional encoding architectures such as transformers or even decision-tree-based models.

REFERENCES

- [1] B. Dai and D. Lin, "Contrastive Learning for Image Captioning," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/46922a0880a8f11f8f69cbb52b1396be-Abstract.html
- [2] B. Sun, B. Li, S. Cai, Y. Yuan, and C. Zhang, "FSCE: Few-Shot Object Detection via Contrastive Proposal Encoding," 2021, pp. 7352–7362. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2021/html/Sun_FSCE_Few-Shot_Object_Detection_via_Contrastive_Proposal_Encoding_CVPR_2021_paper.html
- [3] S. Chopra, R. Hadsell, and Y. LeCun, "Learning a similarity metric discriminatively, with application to face verification," in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, vol. 1, Jun. 2005, pp. 539–546 vol. 1, ISSN: 1063-6919. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/1467314>
- [4] J. Chen, Z. Yuan, J. Peng, L. Chen, H. Huang, J. Zhu, Y. Liu, and H. Li, "DASNet: Dual Attentive Fully Convolutional Siamese Networks for Change Detection in High-Resolution Satellite Images," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 14, pp. 1194–1206, 2021. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9259045>
- [5] G. H. Chang, D. T. Felson, S. Qiu, A. Guermazi, T. D. Capellini, and V. B. Kolachalama, "Assessment of knee pain from MR imaging using a convolutional Siamese network," *European Radiology*, vol. 30, no. 6, pp. 3538–3548, Jun. 2020. [Online]. Available: <https://doi.org/10.1007/s00330-020-06658-3>
- [6] A. Nandy, S. Haldar, S. Banerjee, and S. Mitra, "A Survey on Applications of Siamese Neural Networks in Computer Vision," in *2020 International Conference for Emerging Technology (INCET)*, Jun. 2020, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9153977>
- [7] A. Chatterjee, V. Thambawita, M. A. Riegler, and P. Halvorsen, "Supervised Anomaly Detection in Univariate Time-Series Using 1D Convolutional Siamese Networks," *IEEE Access*, vol. 13, pp. 70 980–71 006, 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10966923>
- [8] D. Fernández-Llaneza, S. Ulander, D. Gogishvili, E. Nittinger, H. Zhao, and C. Tyrchán, "Siamese Recurrent neural network with a self-attention mechanism for bioactivity prediction," *ACS omega*, vol. 6, no. 16, pp. 11 086–11 094, 2021.
- [9] J. Mueller and A. Thyagarajan, "Siamese Recurrent Architectures for Learning Sentence Similarity," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, Mar. 2016. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/10350>
- [10] P. Neculoiu, M. Versteegh, and M. Rotaru, "Learning Text Similarity with Siamese Recurrent Networks," in *Proceedings of the 1st Workshop on Representation Learning for NLP*, P. Blunsom, K. Cho, S. Cohen, E. Grefenstette, K. M. Hermann, L. Rimell, J. Weston, and S. W.-t. Yih, Eds. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 148–157. [Online]. Available: <https://aclanthology.org/W16-1617/>
- [11] L. Zhao, Z. Shang, L. Zhao, T. Zhang, and Y. Y. Tang, "Software defect prediction via cost-sensitive Siamese parallel fully-connected neural networks," *Neurocomputing*, vol. 352, pp. 64–74, Aug. 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231219305004>
- [12] L. Zheng, S. Duffner, K. Idrissi, C. Garcia, and A. Baskurt, "Siamese multi-layer perceptrons for dimensionality reduction and face identification," *Multimedia Tools and Applications*, vol. 75, no. 9, pp. 5055–5073, May 2016. [Online]. Available: <https://doi.org/10.1007/s11042-015-2847-3>
- [13] X. Chen, S. Xie, and K. He, "An Empirical Study of Training Self-Supervised Vision Transformers," 2021, pp. 9640–9649. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2021/html/Chen_An_Empirical_Study_of_Training_Self-Supervised_Vision_Transformers_ICCV_2021_paper.html
- [14] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," Jun. 2021, arXiv:2010.11929 [cs]. [Online]. Available: <http://arxiv.org/abs/2010.11929>
- [15] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised Contrastive Learning," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 18 661–18 673. [Online]. Available: <https://papers.nips.cc/paper/2020/hash/d89a66c7c80a29b1bdbab0f2a1a94af8-Abstract.html>
- [16] P. Billion-Polak and T. M. Khoshgoftaar, "An Evaluation of Low-Shot Learning Techniques for the Detection of Credit Card Fraud," in *2024 International Conference on Machine Learning and Applications (ICMLA)*, Dec. 2024, pp. 304–311. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10903350>
- [17] PyTorch, "Embedding," [Online]. Available: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Embedding.html>
- [18] P. Billion-Polak and T. M. Khoshgoftaar, "Evaluating techniques from low-shot learning on traditional imbalanced classification tasks," *Journal of Big Data*, vol. 12, no. 1, p. 110, May 2025. [Online]. Available: <https://doi.org/10.1186/s40537-025-01171-0>
- [19] J. M. Johnson and T. M. Khoshgoftaar, "Medicare fraud detection using neural networks," *Journal of Big Data*, vol. 6, no. 1, p. 63, Jul. 2019. [Online]. Available: <https://doi.org/10.1186/s40537-019-0225-0>
- [20] Q. Liang, R. A. Bauder, and T. M. Khoshgoftaar, "Enhancing Medicare Fraud Detection: Random Undersampling Followed by SHAP-Driven Feature Selection with Big Data," in *2024 IEEE 36th International Conference on Tools with Artificial Intelligence (ICTAI)*, Oct. 2024, pp. 256–263.
- [21] H. He and E. A. Garcia, "Learning from Imbalanced Data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009, conference Name: IEEE Transactions on Knowledge and Data Engineering. [Online]. Available: <https://ieeexplore.ieee.org/document/5128907>
- [22] N. Japkowicz, "The Class Imbalance Problem: Significance and Strategies," 2000. [Online]. Available: <https://www.semanticscholar.org/paper/The-Class-Imbalance-Problem%3A-Significance-and-Japkowicz/907b02c6322d0e7dff6b0201b03e3d2c6bc1d38f>
- [23] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLOS ONE*, vol. 10, no. 3, pp. 1–21, Mar. 2015. [Online]. Available: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0118432>
- [24] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31. Curran Associates, Inc., 2018. [Online]. Available: <https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html>
- [25] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>
- [26] P. Hajek, M. Z. Abedin, and U. Sivarajah, "Fraud Detection in Mobile Payment Systems using an XGBoost-based Framework," *Information Systems Frontiers*, vol. 25, no. 5, pp. 1985–2003, Oct. 2023. [Online]. Available: <https://doi.org/10.1007/s10796-022-10346-6>
- [27] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" *Advances in Neural Information Processing Systems*, vol. 35, pp. 507–520, Dec. 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/0378c7692da36807bdec87ab043cdadc-Abstract-Datasets_and_Benchmarks.html